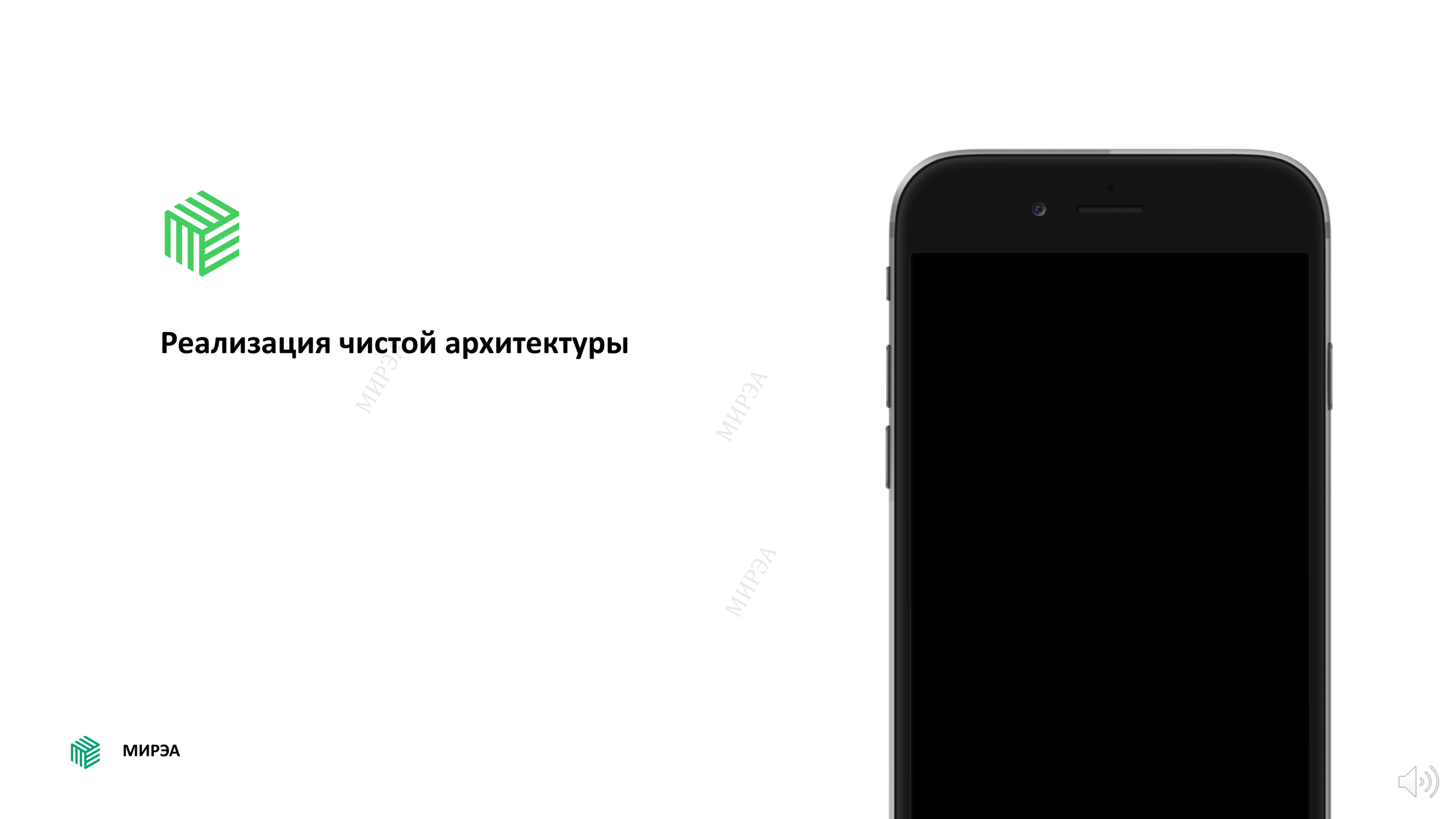
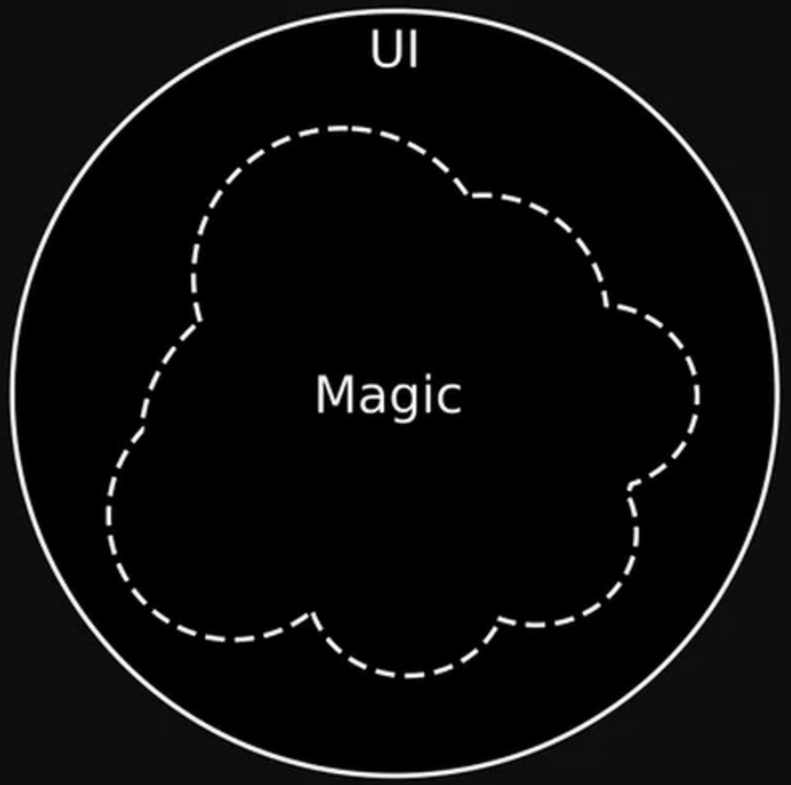


Реализация чистой архитектуры

МИРЭА
МИРЭА
МИРЭА

Структурирование кода



Что видят другие



Что видят студенты МИРЭА

Спагетти???



1. **Трудности с поддержкой и сопровождением**. Любое изменение может потребовать изменений в нескольких частях системы, что увеличивает риск возникновения багов. Разработчики часто тратят больше времени на то, чтобы понять, как работает код, и где именно нужно внести изменения.
2. **Усложнение понимания кода.** Новые разработчики, приходя в проект, сталкиваются с большими сложностями, пытаясь разобраться в логике работы системы.
3. **Сложности с тестированием**. Отсутствие четкой структуры затрудняет изоляцию компонентов для тестирования. В результате тестирование становится более сложным и менее эффективным, что увеличивает вероятность пропуска ошибок. Кроме того, сложно автоматизировать тестирование, если код не структурирован и не разбит на модули.

МИРЭА МИРЭА МИРЭА

Спагетти???



4. Увеличение технического долга. Чем дольше проект развивается без должного внимания к структуре, тем сложнее и дороже становится его поддержка и модернизация. В конечном итоге это может привести к необходимости переписывания значительных частей системы.

5.Низкая производительность команды. Когда код не структурирован, члены команды часто сталкиваются с проблемами при взаимодействии друг с другом. Модули могут перекрываться по функциональности, зависимости могут быть неопределенными или запутанными, что приводит к конфликтам и снижению производительности команды.

6.Затрудненное масштабирование. Неструктурированный код затрудняет добавление новых функций и масштабирование системы. Это затрудняет рост проекта и его адаптацию к новым требованиям.

МИРЭА

Спагетти???



7. **Низкое качество кода.** Без четкой структуры код часто становится запутанным и трудно управляемым. Это приводит к дублированию логики, недостатку ясных интерфейсов и плохой декомпозиции задач.

8.Отсутствие модульности. В неструктурированном коде модули и функции часто переплетены между собой. Если одна часть системы зависит от нескольких других, любое изменение в одном модуле может вызвать каскадный эффект на другие части кода.

9.Проблемы с повторным использованием кода.

10.Проблемы с безопасностью. Когда код не структурирован, может быть трудно реализовать и поддерживать меры безопасности. Например, если логика безопасности распределена по всему коду, а не сконцентрирована в одном месте, это может привести к уязвимостям и сложности в управлении доступом к данным.

МИРЭА МИРЭА МИРЭА

СТРУКТУРИРОВАНИЕ КОДА

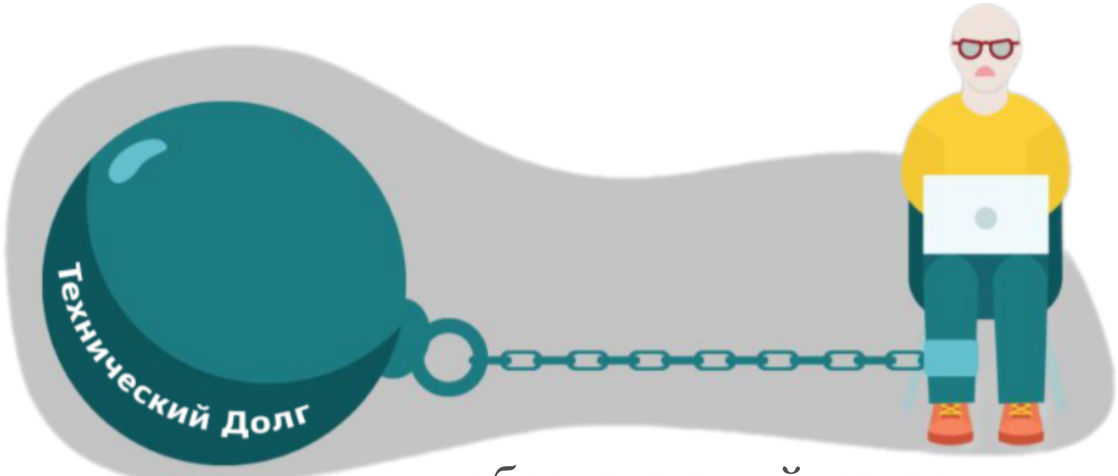
1. **Улучшение читаемости и понимания кода.** Разработчики, работающие над проектом, могут быстро понять, что делает тот или иной модуль, класс или функция, даже если они впервые сталкиваются с кодом. Это особенно важно в командах, где люди часто меняются или работают с разными частями проекта.
2. **Облегчение поддержки и расширения.** Код, который хорошо структурирован, проще поддерживать и расширять. Внесение изменений в такой код требует меньших усилий, поскольку четкое разделение на модули и слои позволяет модифицировать одну часть системы, не затрагивая другие. Это снижает вероятность возникновения ошибок и ускоряет процесс разработки.
3. **Обеспечение модульности.** Структурирование кода подразумевает разделение системы на отдельные модули с четко определенными обязанностями. Модульность позволяет разрабатывать, тестировать и изменять части системы независимо друг от друга. Это повышает гибкость системы и делает её более адаптируемой к изменениям требований или технологий.

МИРЭА МИРЭА МИРЭА МИРЭА

СТРУКТУРИРОВАНИЕ КОДА

4. **Улучшение тестируемости**. Когда код структурирован, его легче тестировать. Модули, отвечающие за разные аспекты системы (например, бизнес-логику, работу с базой данных, взаимодействие с пользователем), могут быть протестированы в изоляции.
5. **Повышение производительности команды**. Хорошо структурированный код способствует улучшению командной работы. Когда каждый член команды понимает, как организован код, и следует единым принципам структурирования, процессы разработки и ревью кода становятся более эффективными.
6. **Упрощение повторного использования кода**. Структурированный код проще переиспользовать. Например, если в проекте имеется модуль, отвечающий за определенную функциональность, его возможно использовать в других проектах без значительных изменений, что способствует созданию библиотек и общих решений, которые возможно использовать многократно.

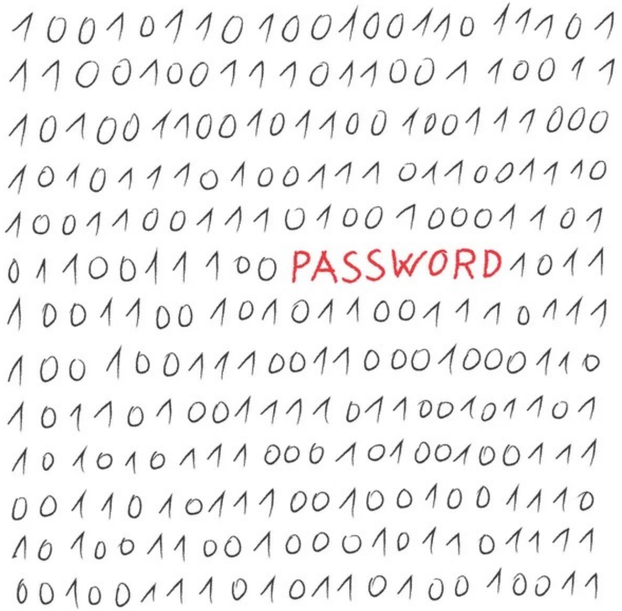
СТРУКТУРИРОВАНИЕ КОДА



7. **Снижение технического долга.** Технический долг — это термин, обозначающий затраты на исправление плохо написанного или неструктурированного кода в будущем. Чем хуже структурирован код, тем больше времени и ресурсов потребуется для его поддержания и модернизации. Хорошее структурирование помогает снизить технический долг, делая код более устойчивым к изменениям и улучшая его качество.
8. **Обеспечение гибкости и масштабируемости.** В условиях растущего проекта структурированный код позволяет легко масштабировать систему. Это достигается благодаря ясной иерархии компонентов и модулей, которые легко интегрируются друг с другом.

МИРЭА

СТРУКТУРИРОВАНИЕ КОДА



9. **Повышение безопасности.** Код, который структурирован и разделен на отдельные слои и модули, позволяет лучше контролировать доступ к данным и функциональности системы. Например, бизнес-логика может быть отделена от пользовательского интерфейса, что предотвращает несанкционированный доступ к важным данным.
10. **Создание долгосрочной архитектуры.** Хорошая структура кода помогает создать архитектуру, которая может быть устойчивой в долгосрочной перспективе. Даже если технология изменяется или требования к системе эволюционируют, хорошо структурированный код позволяет легко адаптироваться к этим изменениям, минимизируя необходимость переписывания значительной части системы.

МИРЭА МИРЭА МИРЭА МИРЭА



Го́вард Ка́ннингем

*«Технический долг похож на **кредит** в банке — если его не гасить, то ваш долг будет расти всё сильнее и сильнее»*

Осмысленное «забивание» на хорошее (в плане поддержки, читабельности, гибкости и пр.) решение в пользу получения краткосрочной выгоды



Роберт Мартин

МИРЭА МИРЭА МИРЭА МИРЭА

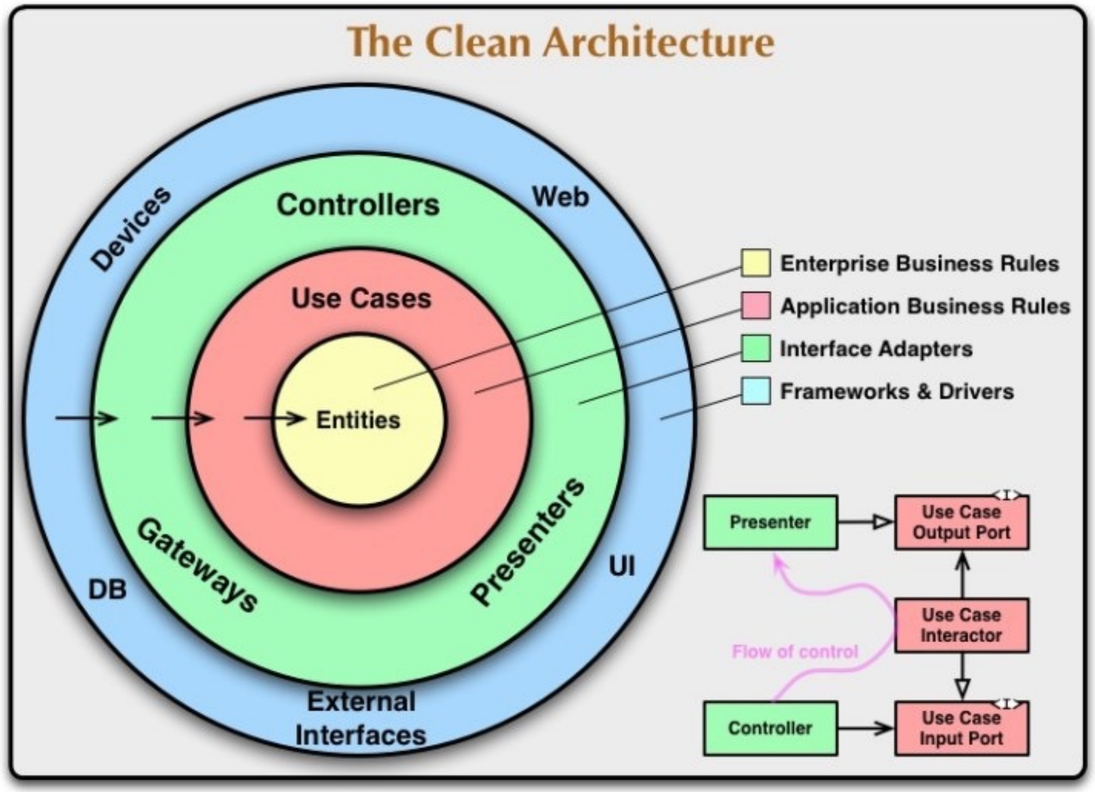
Чистая архитектура

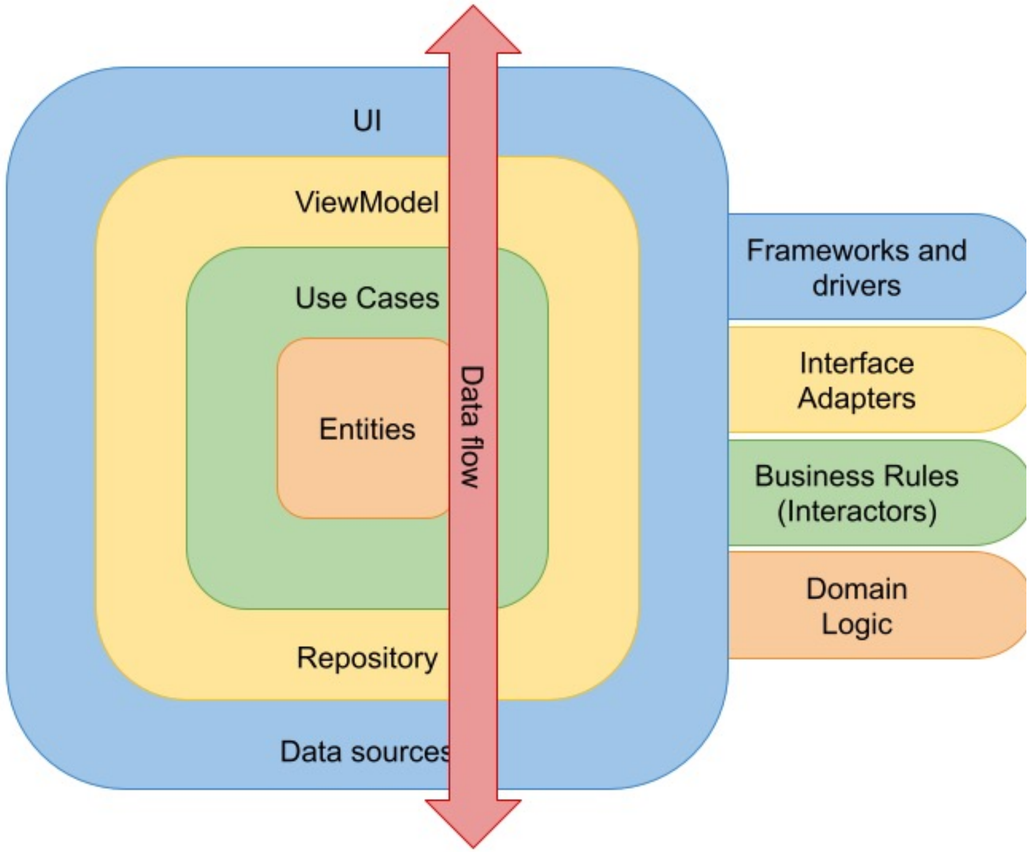
Хорошая архитектура должна позволять писать тесты для классов, содержащих бизнес-логику и должна строить модули приложения независимыми от почти всех внешних элементов.

Качество кода приложения можно даже замерить стандартной единицей измерения – количество WTF в минуту.

(из книги Роберта Мартина “Clean Code”)

Основные понятия чистой архитектуры





Основные понятия чистой архитектуры

1. Слои архитектуры

– **Entities** — центральный слой, содержащий бизнес-логику и правила предметной области. Сущности являются независимыми от внешних изменений, таких как изменения в базе данных или пользовательском интерфейсе.

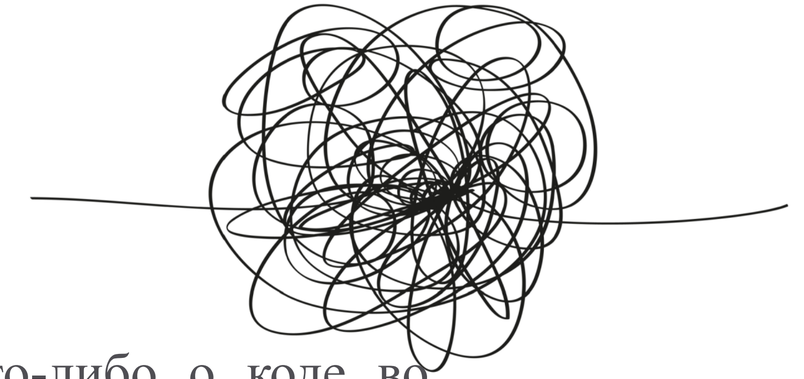
– **Use Cases** — слой содержит логику приложения, которая определяет, как должны использоваться сущности для достижения конкретных целей. Сценарии использования описывают взаимодействие между сущностями.

– **Interface Adapters** — слой преобразует данные из форматов, удобных для использования (например, API или базы данных), в форматы, понятные для внутренней логики приложения, и наоборот. Примеры включают преобразование данных из объектов базы данных в бизнес-объекты и обратно.

– **Frameworks and Drivers** — внешний слой, содержащий фреймворки, инструменты, базы данных и так далее. В данном слое код связывается с предыдущим слоем, но не влияет на внутренние слои.

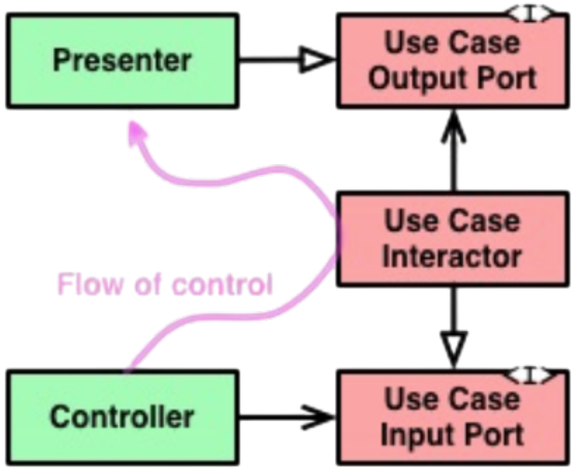
МИРЭА МИРЭА МИРЭА МИРЭА

Основные понятия чистой архитектуры



2. Правило зависимости (The Dependency Rule)

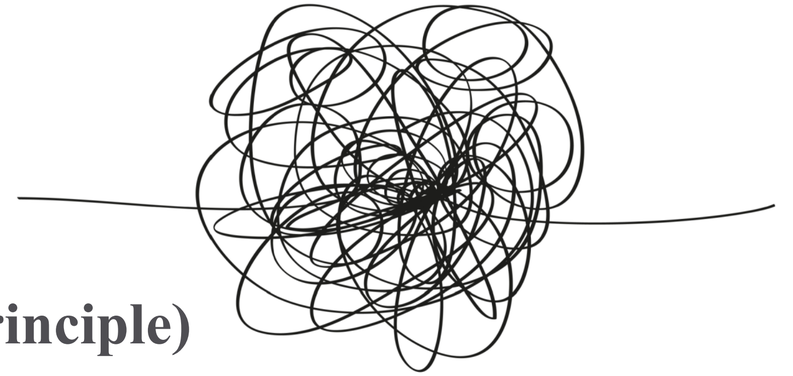
"Никакой исходный код во внутреннем круге не должен знать что-либо о коде во внешнем круге." Это означает, что внутренние слои (например, бизнес-логика) не зависят от конкретных деталей реализации внешних слоев. Чтобы зависимость была направлена в сторону обратную потоку данных, применяется принцип инверсии зависимостей.



3. Границы между слоями и модулями (Boundaries)

Переходы между слоями осуществляются через Boundaries, то есть через два интерфейса: один для запроса и один для ответа. Они нужны, чтобы внутренний слой не зависел от внешнего (следуя Dependency Rule), но при этом мог передать ему данные.

Основные понятия чистой архитектуры



4. **Принцип инверсии зависимостей (Dependency Inversion Principle)**

Зависимости направлены от внешних слоев к внутренним. Внутренние слои не зависят от внешних слоев, а наоборот, внешние слои зависят от интерфейсов, определенных во внутренних слоях. Это достигается с помощью Dependency Injection, что позволяет легко подменять реализации зависимостей (например, для тестирования).

5. **Тестируемость**. За счет строгого соблюдения принципов разделения слоев и инверсии зависимостей, чистая архитектура делает код легко тестируемым. Отдельные слои и модули возможно тестировать независимо друг от друга, используя заглушки (mock objects) и другие техники.

МИРЭА МИРЭА МИРЭА МИРЭА



Основные понятия чистой архитектуры

6. **Независимость от UI (Independence from UI)**. Чистая архитектура не зависит от конкретных реализаций пользовательского интерфейса. UI является просто еще одним внешним слоем, который может быть заменен или модифицирован без необходимости изменять бизнес-логику.

7. **Независимость от базы данных (Independence from Database):** База данных также считается деталью реализации и не должна влиять на внутреннюю бизнес-логику. Это позволяет легко заменять одну базу данных на другую или даже поддерживать несколько баз данных.





Основные понятия чистой архитектуры

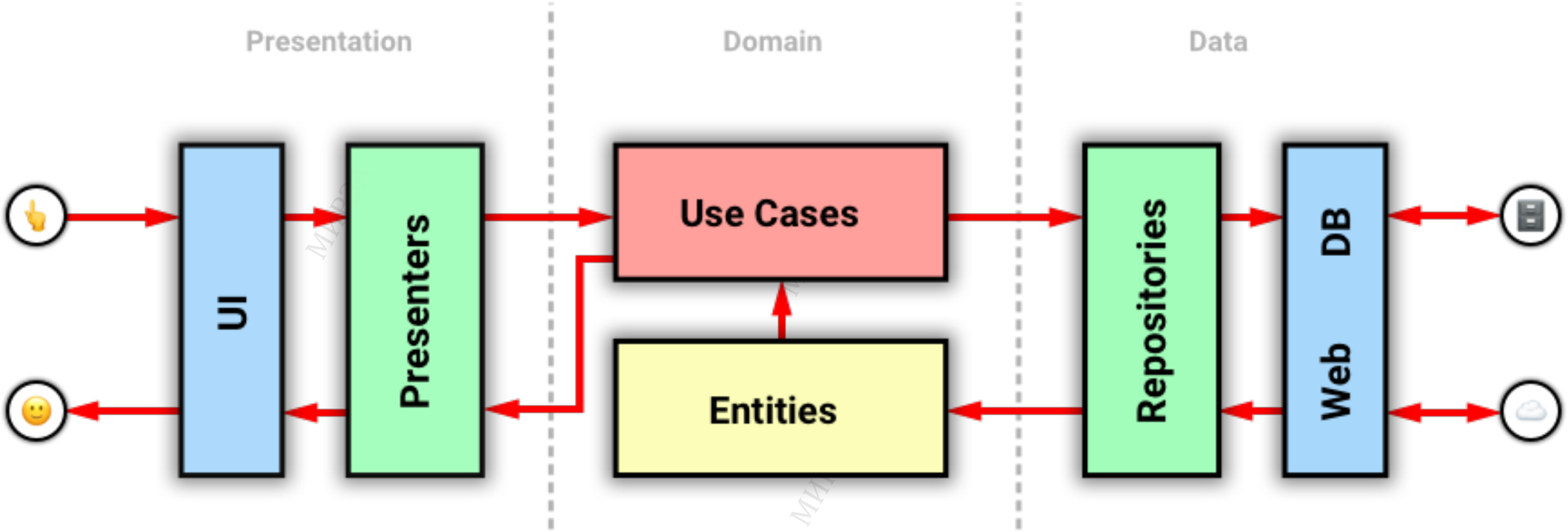


8. **Независимость от внешних агентов (Independence from External Agencies)**. Внешние системы, такие как веб-сервисы, библиотеки или другие модули, также должны быть отделены от бизнес-логики. Это позволяет взаимодействовать с этими системами через адаптеры, которые можно легко заменить или обновить.

9. **Согласованность (Consistency).** Важно, чтобы вся система следовала единому стилю и подходу, что способствует лучшей читаемости, поддерживаемости и обучению новых членов команды.

МИРЭА

Схема разложения слоев по категориям



Entities

– Entities – функции или объекты с методами, которые реализуют логику бизнеса, общую для многих приложений (а если бизнеса нет, то самую высокоуровневую логику приложения);

– DTO, необходимые для работы и перехода между слоями.

Кроме того, когда приложение отдельное, то надо стараться находить и выделять в Entities высокоуровневую логику из слоя UseCases, где зачастую она находится по ошибке.

```
public class Earthquake {
     private String title;
     private Double magnitude;
     private Date time;
     private AlertLevel alertLevel;
     private Location location;
     private String url;
...
 }
```

UseCase и/или Interactor

- Use case – это детализация, описание действия, которое может совершить пользователь системы.
- Interactor – объект, который реализует use case (сценарий использования), используя бизнес-объекты (Entities).

Create Order

Data:

<Customer-id>, <Customer-contact-info>,
<Shipment-destination>, <Shipment-mechanism>,
<Payment-information>

Primary Course:

1. **Order clerk issues "Create Order" command with above data.**
2. **System validates all data.**
3. **System creates order and determines order-id.**
4. **System delivers order-id to clerk.**

Exception Course: Validation Error

1. **System delivers error message to clerk**

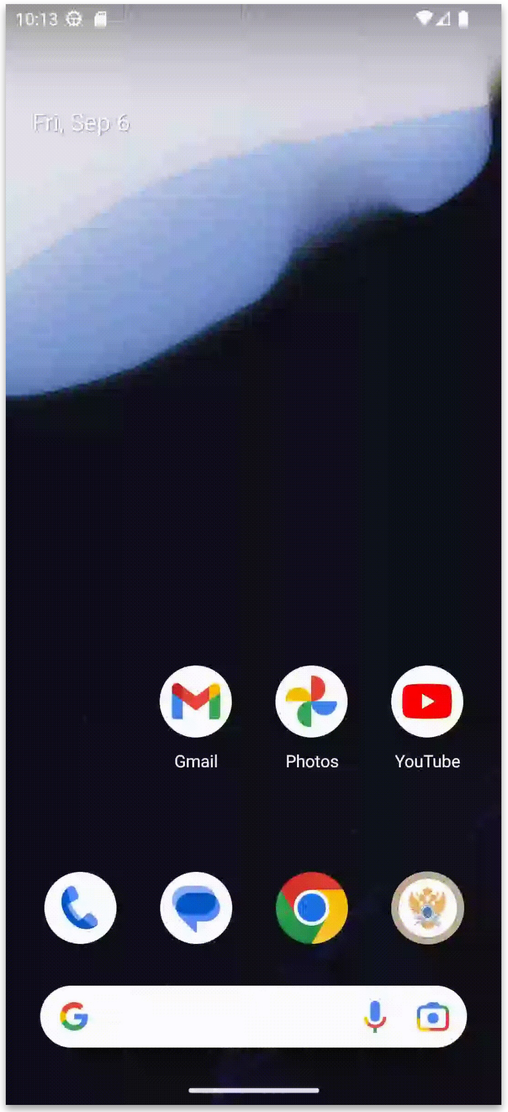
UseCase

– **Определение бизнес-логики**. Какие задачи должен выполнять пользователь? Каковы требования к функциональности? Какие данные используются или изменяются?

– **Идентификация сущностей.** Пример: Для сценария «Оформление заказа» сущностями могут быть «Пользователь», «Товар», «Корзина» и «Заказ».

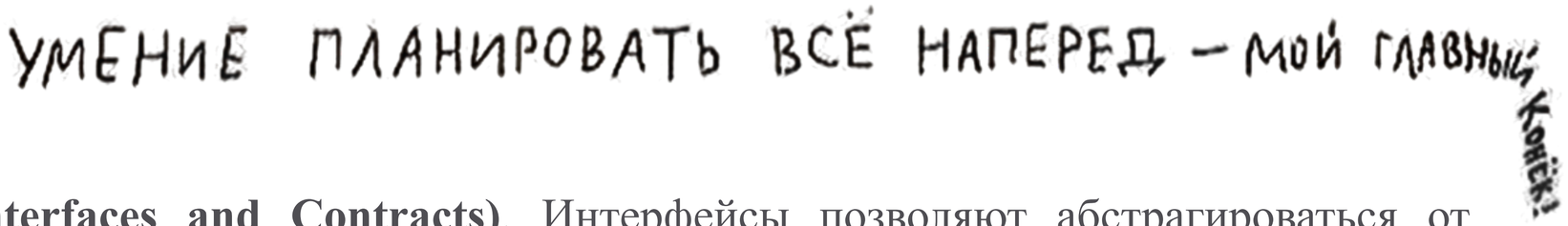
– **Определение границ (Boundaries).** Какие данные должны быть получены из внешних систем (например, API, базы данных)? Как будет происходить обмен данными между слоями?

– **Создание сценариев (Use Case Flow).** Пример: Для «Оформления заказа» шаги могут быть следующими: Пользователь выбирает товар в корзину ⏩ Система проверяет наличие товара ⏩ Пользователь вводит данные о доставке и оплачивает заказ ⏩ Система подтверждает заказ и отправляет уведомление.



МИРЭА

UseCase

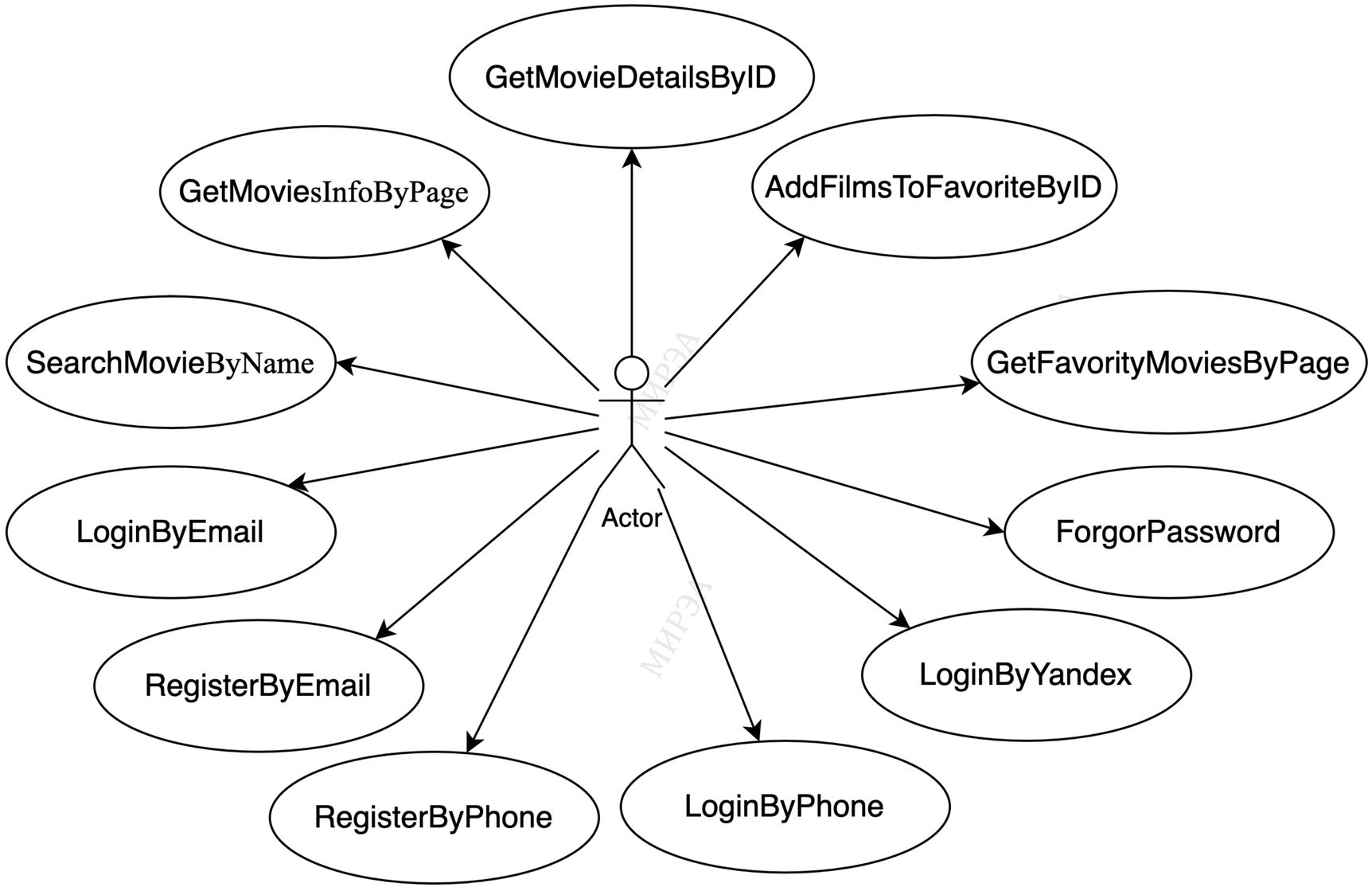


- **Интерфейсы и контракты (Interfaces and Contracts)**. Интерфейсы позволяют абстрагироваться от конкретной реализации и упрощают тестирование. Какие методы или функции необходимы для выполнения сценария? Какие параметры и возвращаемые значения требуются? Пример: В сценарии «Оформление заказа» может быть интерфейс для взаимодействия с платежной системой (например, PaymentGateway), который будет содержать метод processPayment().
- **Обработка ошибок и исключений.** Какие возможны исключения? Как система должна реагировать на исключения? Какие данные или сообщения нужно вернуть пользователю? Пример: В сценарии «Оформление заказа» может быть проверка на наличие товара перед подтверждением заказа. Если товар отсутствует, сценарий должен вернуть сообщение об ошибке пользователю.
- **Тестируемость сценария использования.** Как можно изолировать сценарий для тестирования? Какие зависимости нужно подменить заглушками?

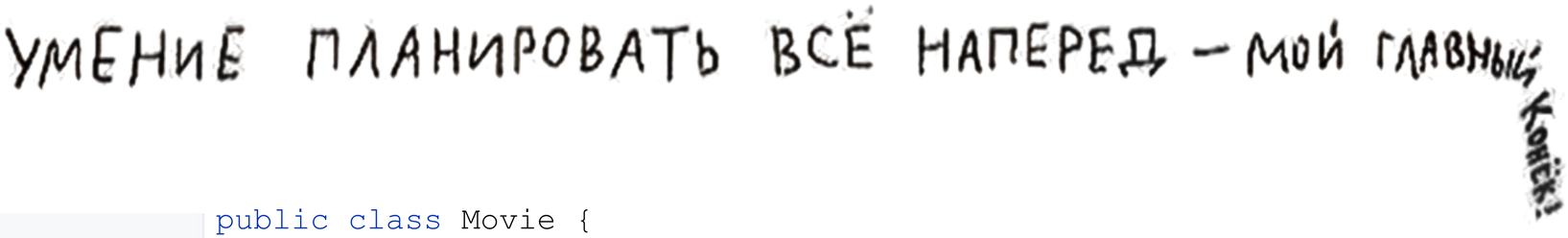
МИРЭА МИРЭА МИРЭА МИРЭА

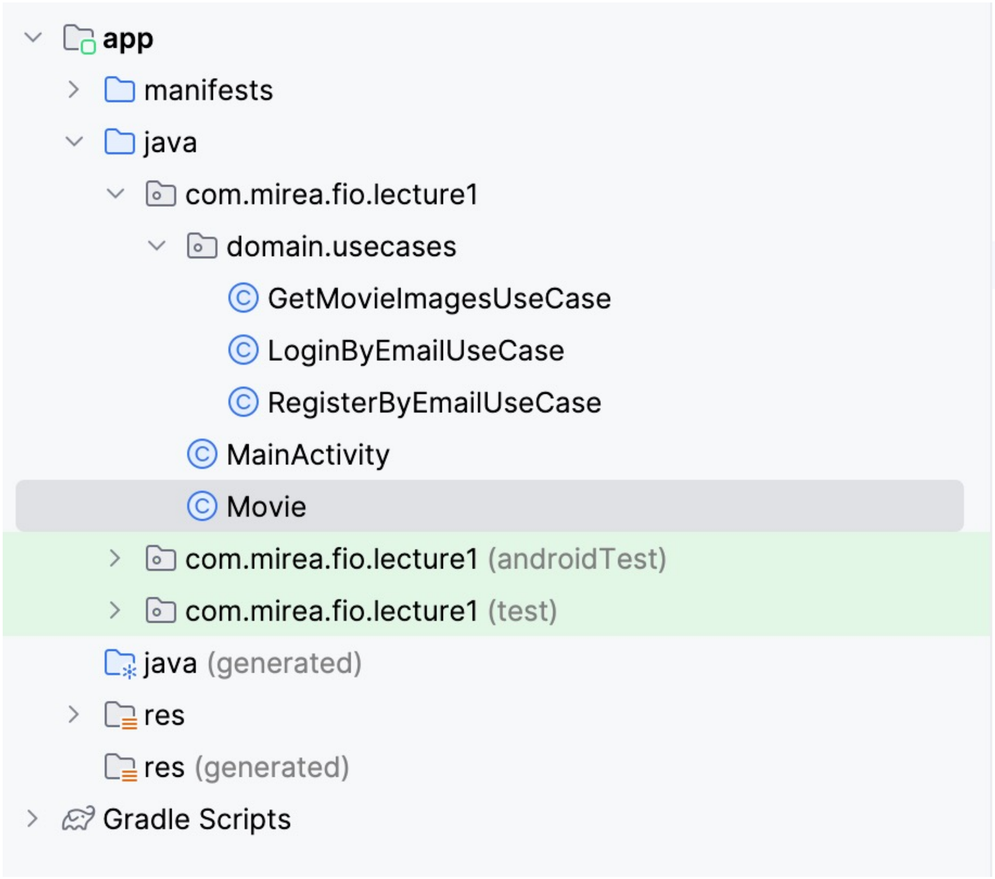
UseCase

УМЕНИЕ ПЛАНИРОВАТЬ ВСЁ НАПЕРЕД – МОЙ ГЛАВНЫЙ КОНЁК!



UseCase






```java
public class Movie {
    private int id;
    private String name;

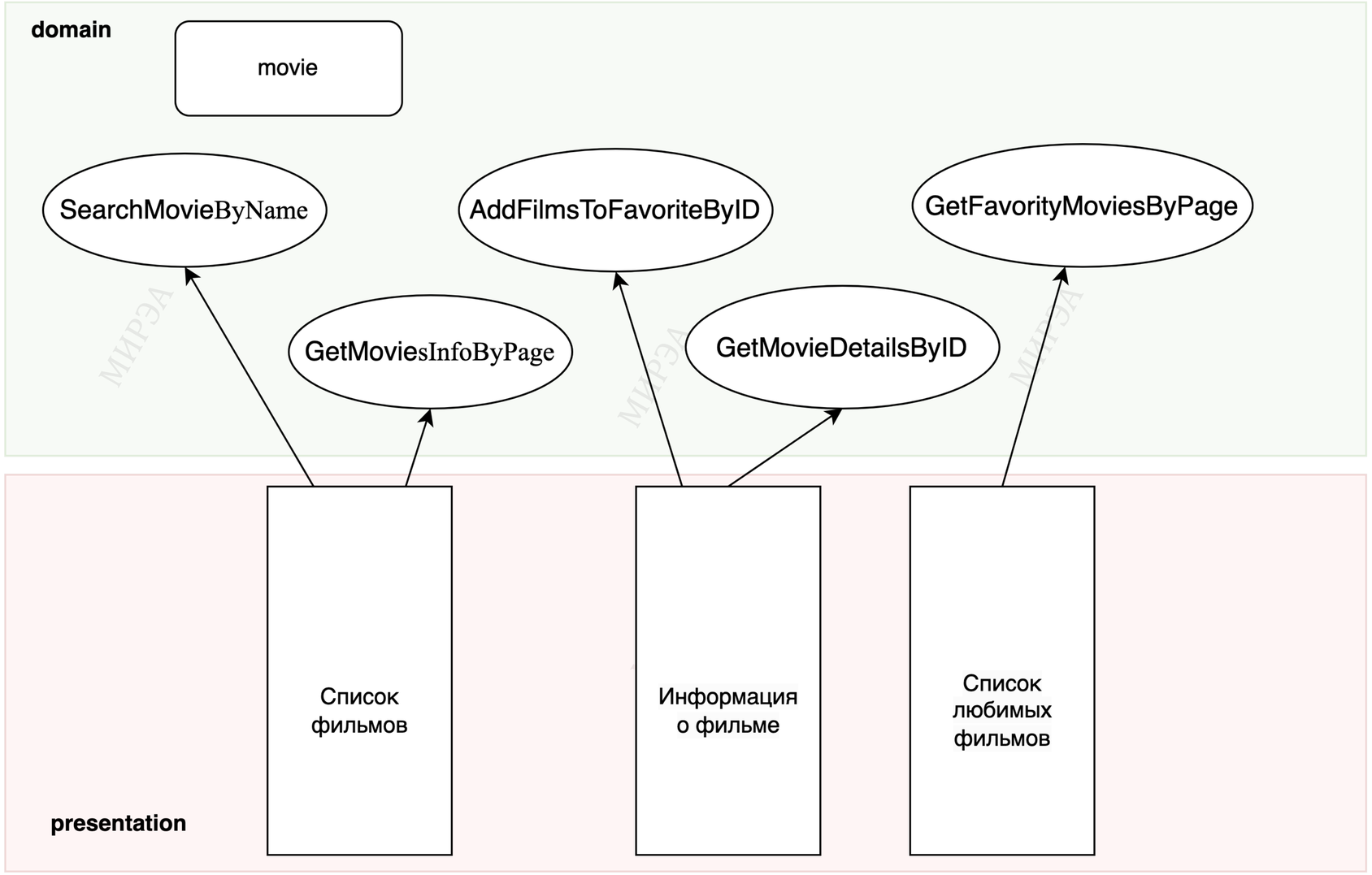
    public Movie(int id, String name) {
        this.id = id;
        this.name = name;
    }
    public String getName() {
        return name;
    }
}

public class GetMoviesInfoByPageUseCase {
    public List<Movie> execute(){
        return Arrays.asList(new Movie(1, "spider man"),
                new Movie(2, "iron man"));
    }
}

List<Movie> moviesList = new GetMoviesInfoByPageUseCase().execute();
for (Movie movie : moviesList) {
    Log.d(MainActivity.class.getSimpleName(), movie.getName());
}
```

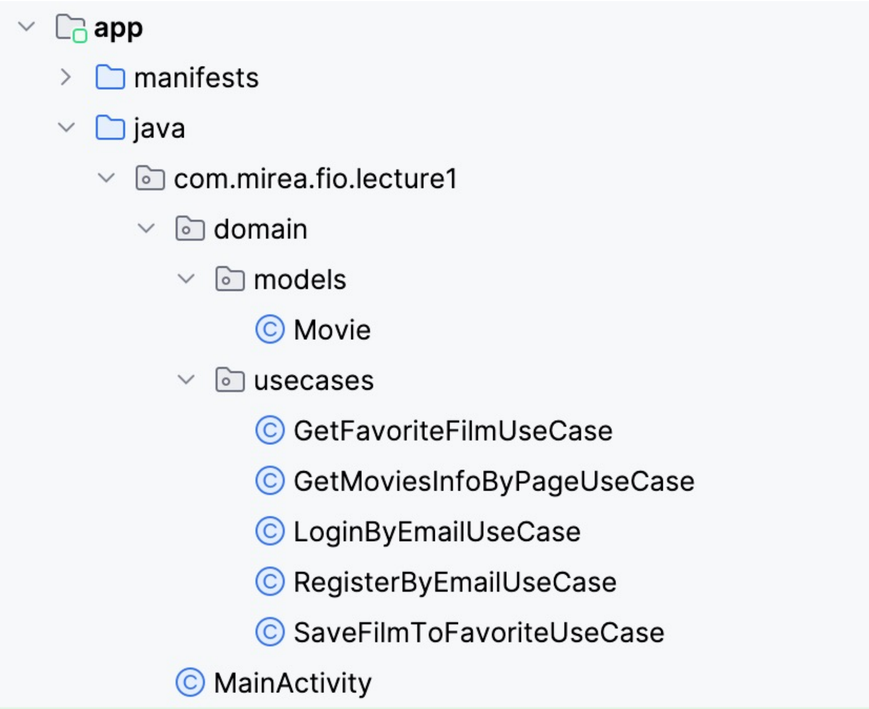
UseCase

УМЕНИЕ ПЛАНИРОВАТЬ ВСЁ НАПЕРЕД — МОЙ ГЛА



UseCase

УМЕНИЕ ПЛАНИРОВАТЬ ВСЁ НАПЕРЕД – МОЙ ГЛА



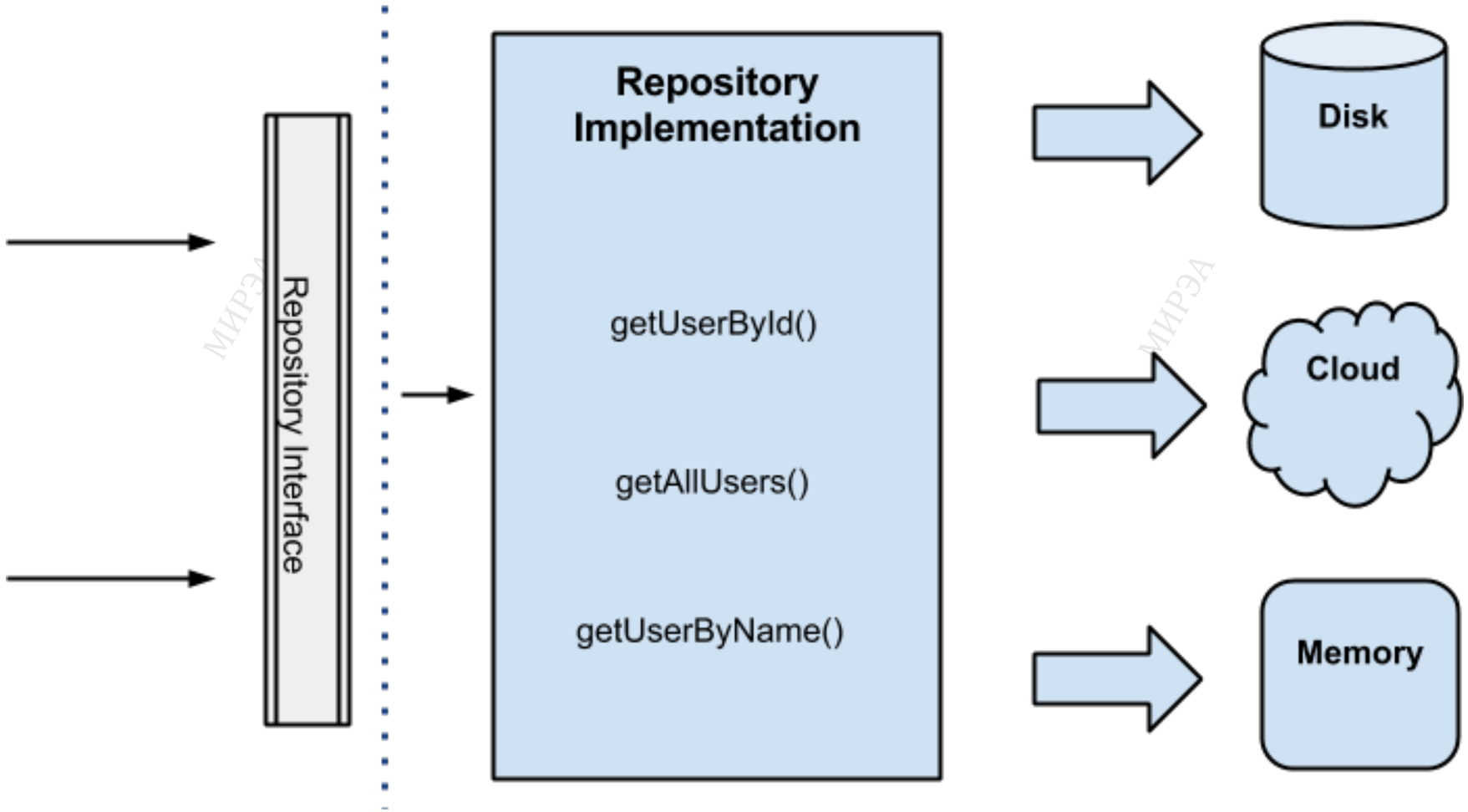

```java
public class GetFavoriteFilmUseCase {

    public Movie execute(){
        return new Movie(3, "Game of thrones");
    }
}
```

```java
public class SaveFilmToFavoriteUseCase {

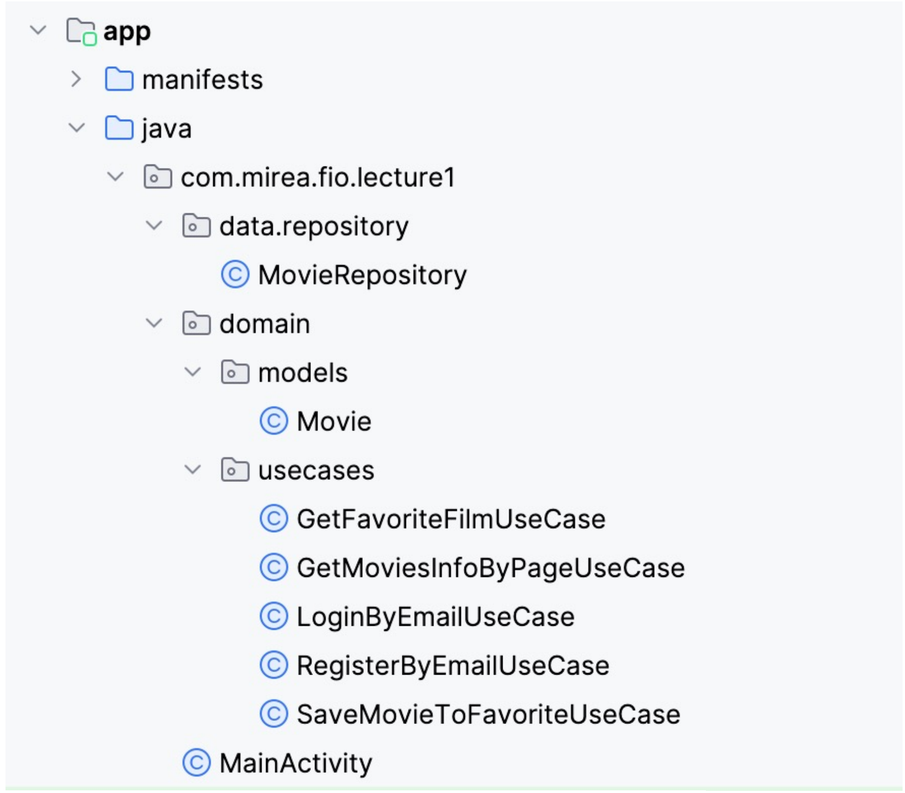
    public boolean execute(Movie movie){
        if (movie.getName().isEmpty()){
            return false;
        }else {
            return true;
        }
    }
}
```

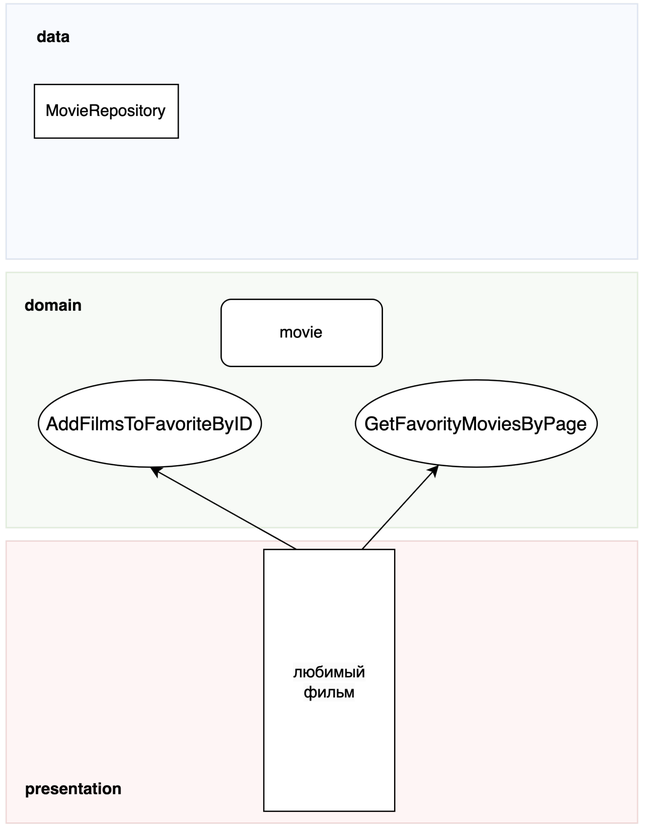
Repository/Gateway



Чистая архитектура

Repository/Gateway





МИРЭА

```
public class MovieRepository {

    public boolean saveMovie(Movie movie){
        return true;
    }

    public Movie getMovie(){
        return new Movie(1,"Doctor Strange");
    }
}
```

Repository/Gateway

```
public class SaveMovieToFavoriteUseCase {
    private MovieRepository movieRepository;

    public SaveMovieToFavoriteUseCase(MovieRepository movieRepository) {
        this.movieRepository = movieRepository;
    }

    public boolean execute(Movie movie){
        return movieRepository.saveMovie(movie);
    }
}
```

